HashiCorp Nomad



Nomad



HashiCorp Nomad

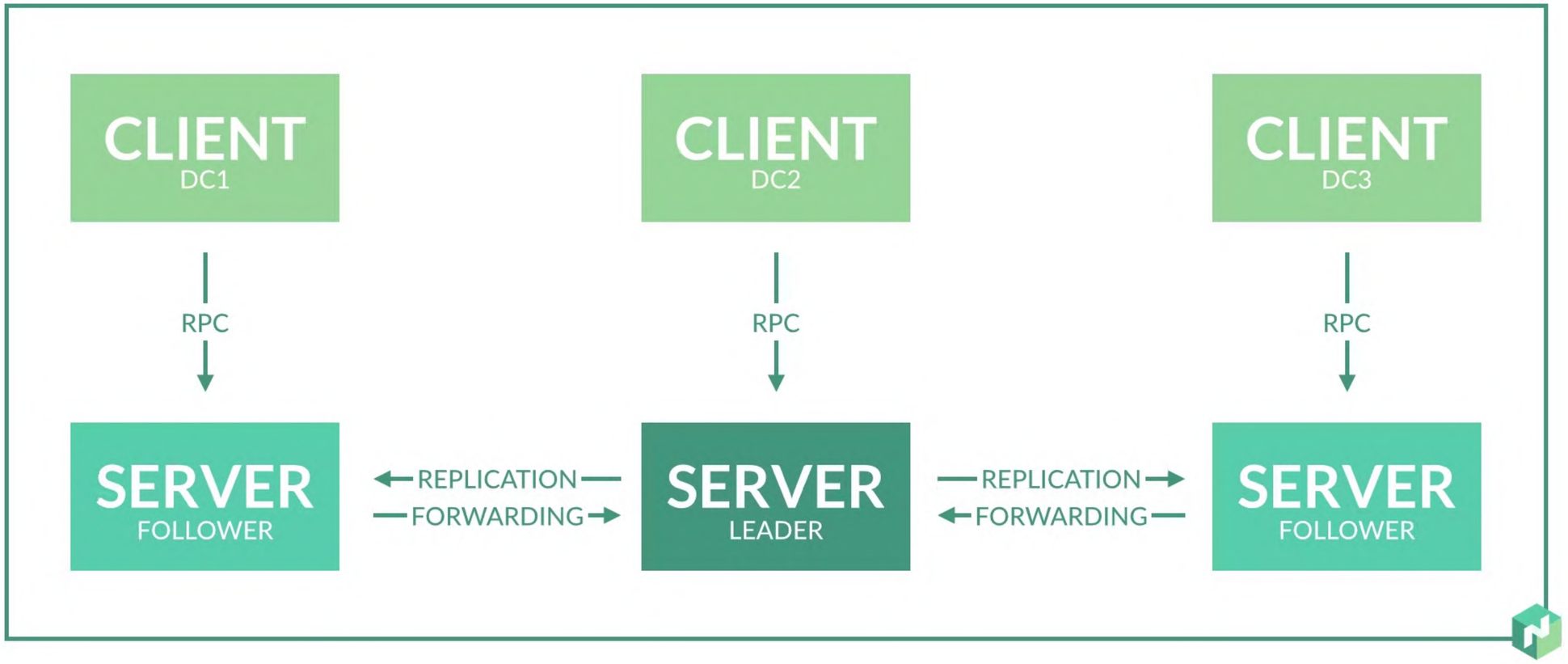
HashiCorp Nomad is an open-source cluster manager and scheduler designed to orchestrate containerized and non-containerized applications across a distributed infrastructure. It is developed by HashiCorp, a company well-known for its other popular tools such as Terraform, Vault, and Consul.

Nomad

Key features of HashiCorp Nomad:

1. **Job Scheduling**: Nomad allows you to define jobs that specify the tasks to be executed, such as running containers, running standalone processes, or managing virtual machines. These jobs are then scheduled and placed on available resources within the cluster.
2. **Multi-Platform Support**: Nomad is platform-agnostic and supports various deployment targets, including Docker containers, VMs, and even standalone executables.
3. **Scaling and Auto-Scaling**: Nomad can automatically scale applications based on resource usage or custom metrics, ensuring the right number of instances to meet the required performance.
4. **High Availability**: Nomad is designed to be highly available, meaning it can tolerate node failures and continue to operate without major disruptions.
5. **Service Discovery and Load Balancing**: It integrates with HashiCorp Consul for service discovery and load balancing, making it easy for applications to locate and communicate with each other.
6. **Declarative Configuration**: You define the desired state of your infrastructure using a declarative configuration file, and Nomad handles the orchestration and resource allocation to achieve that state.
7. **Distributed Scheduling**: Nomad uses a distributed scheduler that optimizes resource allocation across the cluster, ensuring efficient use of resources.
8. **Extensibility**: Nomad is built with a plugin-based architecture, allowing developers to extend and customize its functionality.

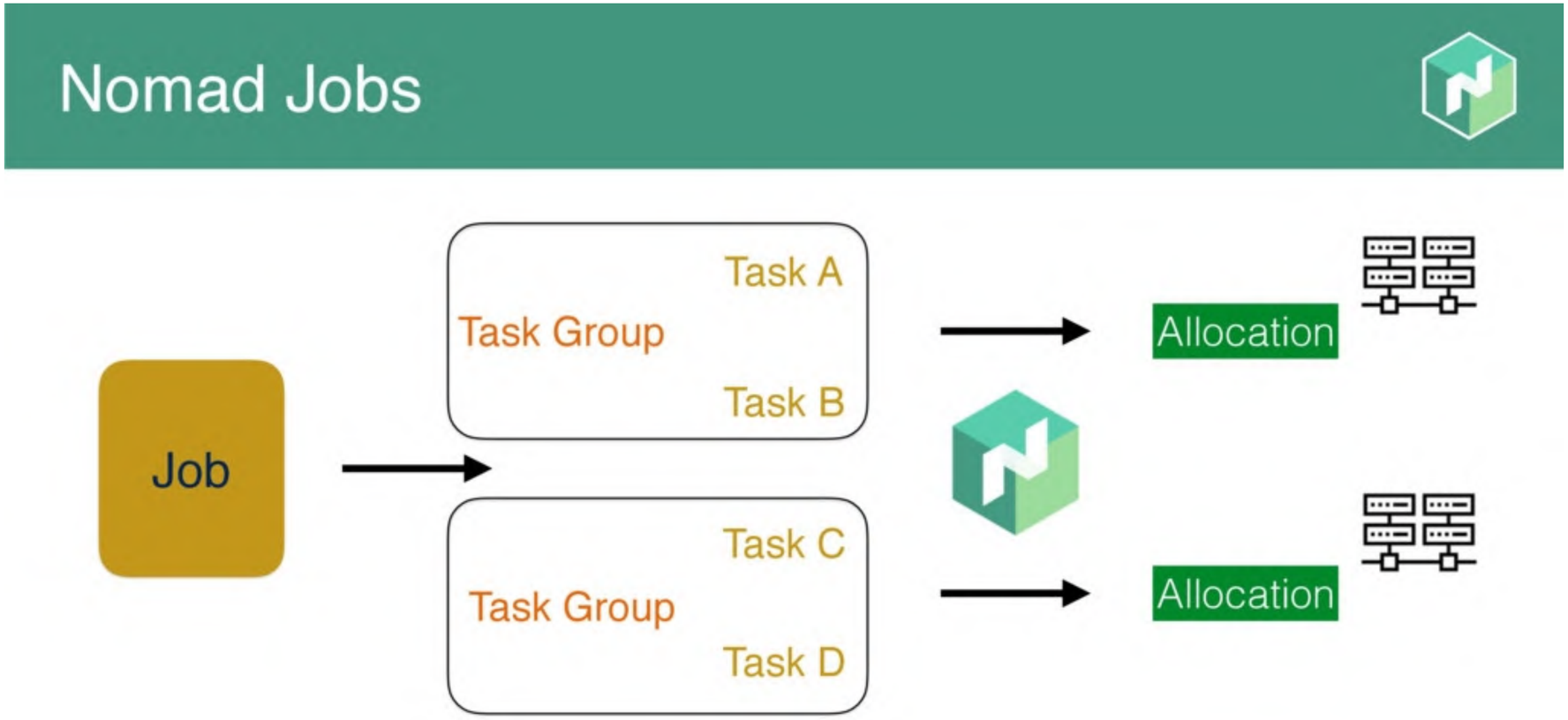
Architecture



Nomad Jobs

- **Job Manifests** are plain-text files that describe how to deploy and manage a service including containers, network access, autoscaling, upgrade strategy, and sidecars! You can think of this like a specification for how to run your app.
- **Groups** are collections of related containers that should be scheduled together (similar to a Kubernetes Pod). Groups of containers are scaled up and down together on the same VMs. Example: Your main API container + monitoring sidecar + Startup Task.
- **Tasks** are Docker Containers (other types are also possible). Nomad Tasks can specify a Docker image, startup command, dynamic configuration files, and CPU + Memory allocation.
- **Services** expose a Task to incoming network traffic. Services are registered in Consul. Service can scale based on CPU + Memory utilization as well as custom queries.
- **Constraints** (and Affinity) allow you to schedule apps on specific Compute Clients within a Nomad Cluster. Think: CPU Architecture, has GPU, AWS Region, Docker version, etc.

Nomad Jobs

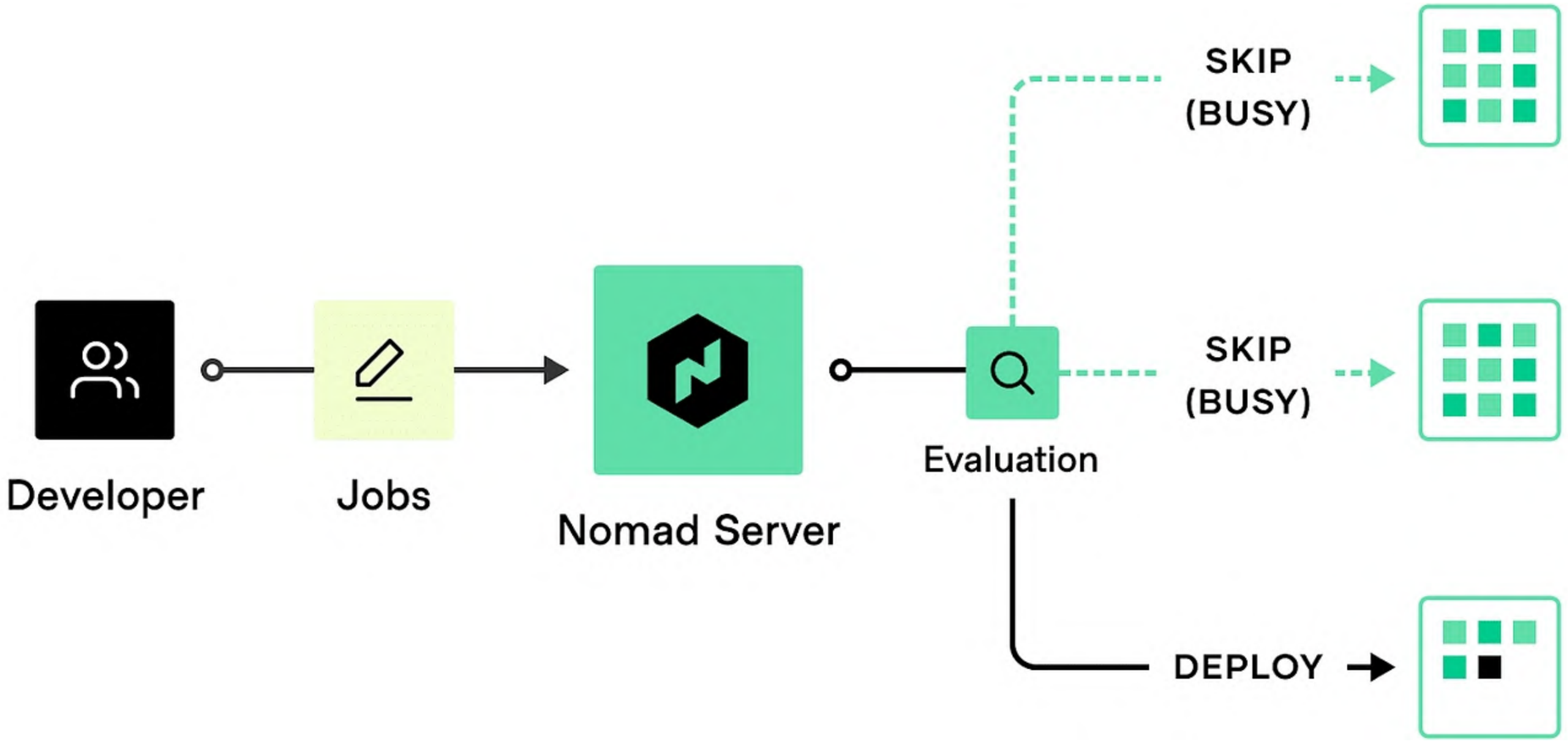

Nomad Jobs
Job
Task Group
Task A
Task B
Task Group
Task C
Task D
Allocation
Allocation

Distributed Scheduling

Distributed scheduling refers to the process of allocating and managing tasks across multiple nodes or resources in a distributed computing environment. In such a system, tasks or jobs are distributed among various computing nodes to achieve better performance, resource utilization, and scalability. It is commonly used in large-scale computing systems, such as cloud computing platforms and data centers, where there are numerous interconnected nodes working together to execute tasks.

The primary goal of distributed scheduling is to optimize the utilization of resources, reduce latency, and balance the workload among different nodes.

Nomad Scheduling



Nomad Scheduling

Nomad has four scheduler types that can be used when creating a job:

1. **Service**: Service scheduler is designed for scheduling long-lived services that should never go down.
2. **Batch**: Batch jobs are much less sensitive to short-term performance fluctuations and are short-lived, finishing in a few minutes to a few days.
3. **System**: The system scheduler is used to register jobs that should be run on all clients that meet the job's constraints. The system scheduler is also invoked when clients join the cluster or transition into the ready state.
4. **System Batch**: The system batch scheduler is used to register jobs that should be run to completion on all clients that meet the job's constraints. The system batch scheduler will schedule jobs similarly to the system scheduler, but like a batch job once a task exits successfully it is not restarted on that client.

HCL

HashiCorp Configuration Language (HCL) is a declarative language developed by HashiCorp, a company that specializes in creating tools for cloud infrastructure automation and management. HCL is primarily used to write configurations for various HashiCorp products, including Terraform, Consul, Vault, Nomad, and others. The language is designed to be easy to read and write, aiming for a human-friendly syntax.

HCL is mainly used to define infrastructure as code (IaC) for provisioning and managing resources in cloud environments. Terraform, in particular, is a popular HashiCorp tool that uses HCL to describe infrastructure resources, providers, and their relationships. It allows users to define the desired state of their infrastructure in configuration files, and Terraform takes care of creating or modifying resources to match that state.

HCL

```
job "example" {
  datacenters = ["dc1"]

  group "cache" {
    network {
      port "db" {
        to = 6379
      }
    }

    task "redis" {
      driver = "docker"

      config {
        image          = "redis:7"
        ports          = ["db"]
        auth_soft_fail = true
      }

      resources {
        cpu    = 500
        memory = 256
      }
    }
  }
}
```

example.nomad hosted with ❤ by GitHub view raw

HashiCorp Consul



HashiCorp Consul is a popular open-source tool developed by HashiCorp, designed for service networking and service discovery. It is part of the HashiCorp product suite, which also includes other tools like Terraform, Vault, and Nomad.

Consul

Consul provides a distributed and highly available solution for discovering and managing services in modern dynamic infrastructure environments, such as cloud-based, containerized, or microservices-based systems. Its primary features include:

1. **Service Discovery**: Consul allows services to register themselves and provides a DNS or HTTP API interface to discover the available services and their locations. This makes it easier for applications to locate and communicate with each other dynamically.
2. **Health Checking**: Consul can monitor the health of services by regularly sending health checks. If a service fails the health check, Consul can automatically remove it from the list of available services, ensuring that only healthy instances are used for communication.
3. **Key-Value Store**: Consul provides a distributed key-value store that can be used for dynamic configuration and other purposes, enabling you to store and retrieve data that is accessible across the entire cluster.
4. **Multi-Datacenter Support**: Consul is designed to work seamlessly across multiple data centers, making it suitable for globally distributed applications.
5. **Secure Communication**: Consul supports various security features like transport layer encryption and access control lists (ACLs) to ensure secure communication and data protection.
6. **Consul Connect**: It offers a service mesh feature called Consul Connect, which allows secure and encrypted communication between services using mutual TLS authentication.
7. **Consul Federation**: This feature enables multiple Consul clusters to communicate with each other, allowing seamless service discovery and access across multiple environments.

Service Discovery

Service discovery is a crucial aspect of distributed computing and networking, particularly in modern software architectures like microservices. It refers to the process of automatically identifying and locating available services in a network so that they can communicate with each other efficiently.

In a distributed system, services are individual components or applications that provide specific functionalities. These services may be running on different machines or containers across the network. Service discovery enables these services to find each other without having to hardcode the network addresses or configurations manually, which can become cumbersome and error-prone as the system scales.

Service discovery plays a fundamental role in enabling dynamic scalability, fault tolerance, and load balancing in modern distributed systems. It allows new services to join the network effortlessly, and if a service fails or needs to be replaced, the service discovery mechanism ensures that other services can still find and communicate with the updated or alternative instances.

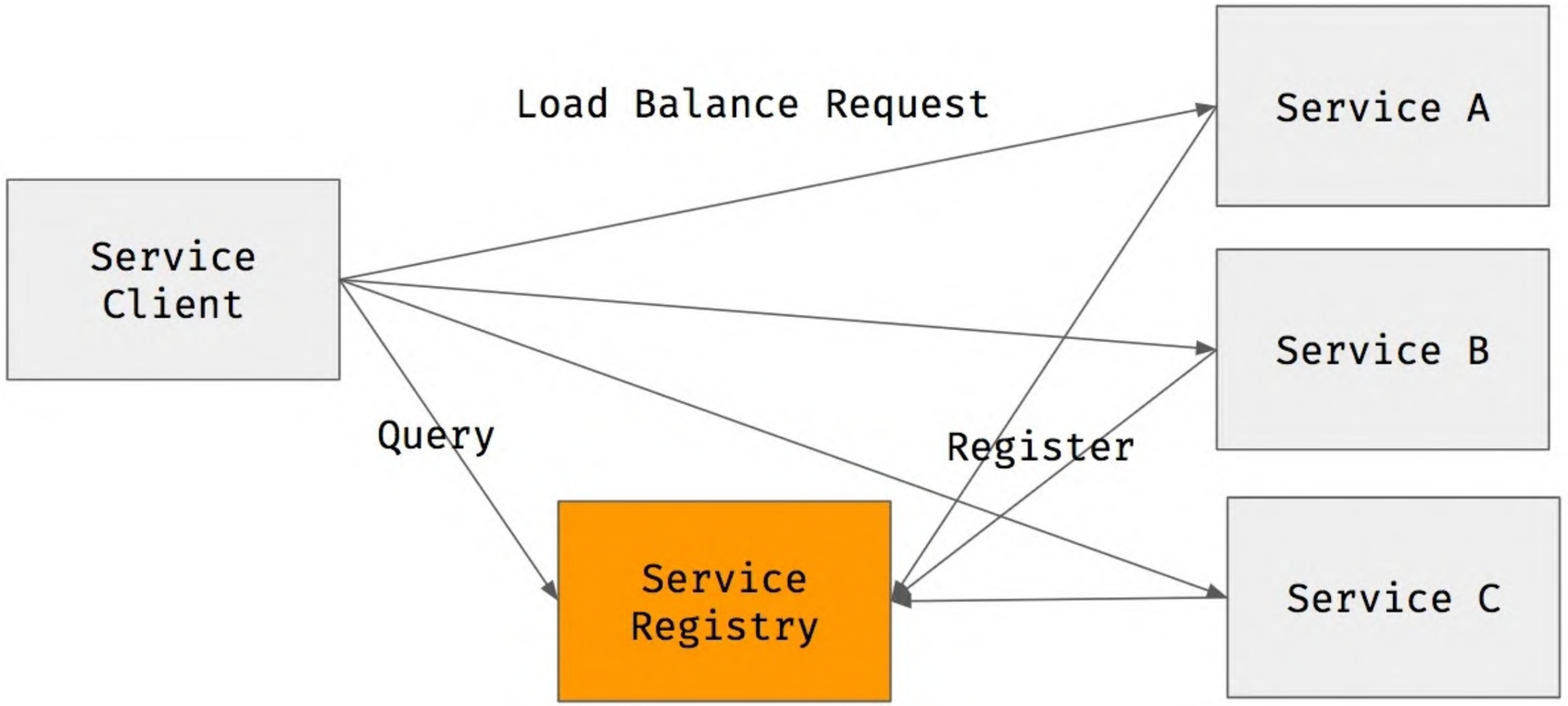
Service Discovery

The traditional approaches for service discovery include:

1. **DNS-Based Service Discovery**: Domain Name System (DNS) can be used to map service names to their corresponding IP addresses. Clients can use DNS queries to discover the IP address of the service they wish to communicate with.
2. **Static Configuration**: Although not automated, static configuration involves maintaining a list of available services and their endpoints in a centralized configuration file. While simple, this method is less flexible and requires manual updates as the service infrastructure changes.
3. **Service Registry**: In a service registry, each service instance registers itself with a central server upon startup. The registry maintains an up-to-date list of available services and their locations. Clients can query the registry to discover the services they need.
4. **Agent-Based Service Discovery**: This approach uses lightweight agents running on each machine where services are hosted. These agents communicate with each other and maintain an updated list of available services on the network.

Some popular service discovery tools and platforms include Consul, etcd, ZooKeeper, and Kubernetes (which provides built-in service discovery features).

Service Registry



Load Balancing

Load balancing is a technique used in computer networking and distributed systems to distribute incoming network traffic or computational workloads across multiple servers, devices, or resources. The primary goal of load balancing is to optimize resource utilization, increase efficiency, and ensure high availability and reliability of the services provided by these resources.

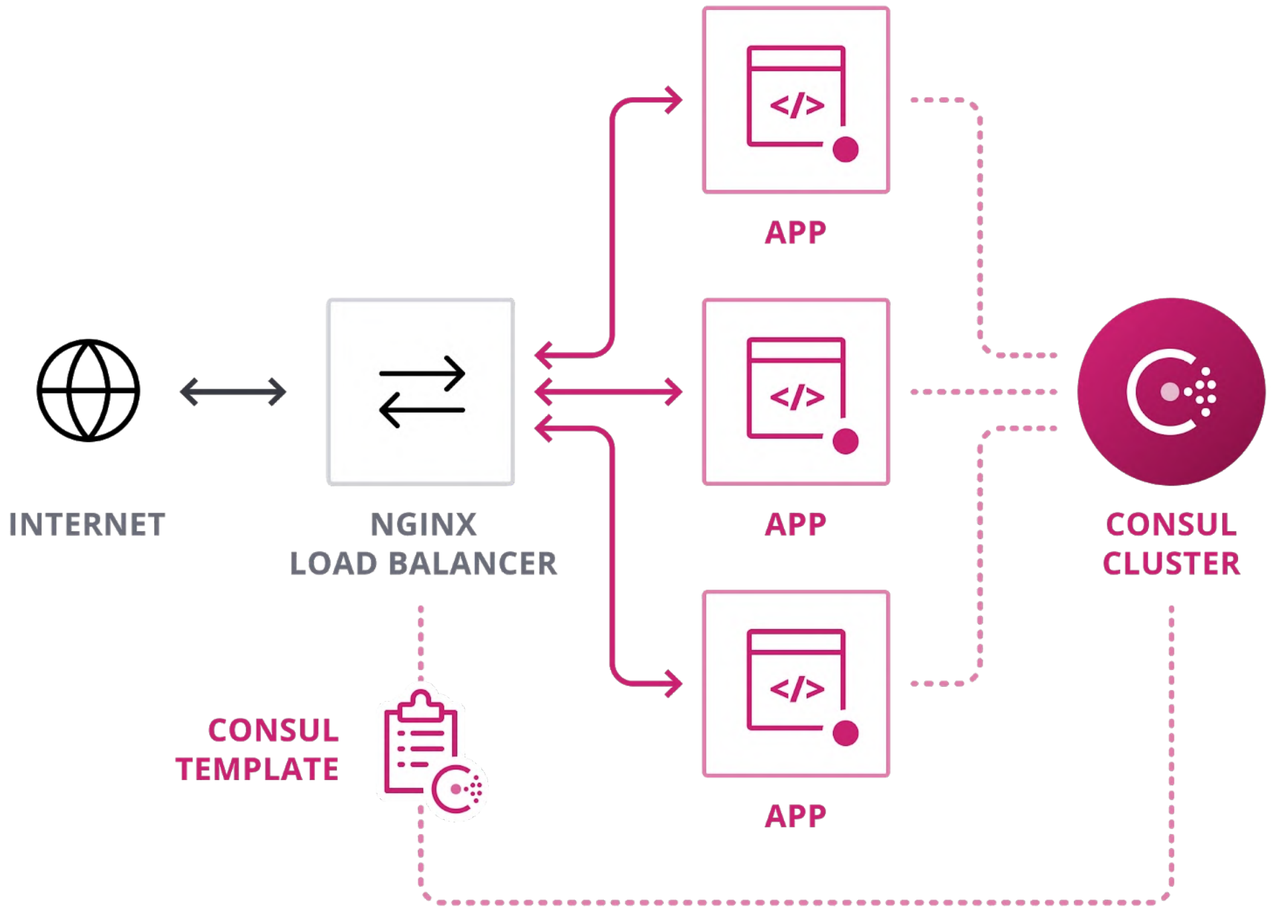
When a server or resource becomes overloaded due to increased traffic or workload, it may slow down or become unresponsive, leading to reduced performance and potential service disruptions. Load balancing helps prevent these issues by evenly distributing the load across available resources, ensuring that no single resource is overwhelmed while others remain underutilized.

Load Balancing

There are various types of load balancing techniques, including:

1. **Local Load Balancing**: This type of load balancing occurs within a single device or server that has multiple processing units, such as CPU cores. It distributes incoming tasks or processes among these cores to utilize them efficiently.
2. **Global Load Balancing**: In a distributed system with multiple servers or data centers located in different geographical regions, global load balancing is used to distribute incoming traffic across these locations. This helps optimize response times and ensures redundancy and fault tolerance.
3. **DNS Load Balancing**: This technique involves using DNS (Domain Name System) to distribute incoming requests across multiple server IP addresses associated with a single domain name. DNS servers respond to queries with different IP addresses, rotating the selection of servers and effectively balancing the load.
4. **Application Load Balancing**: Application-level load balancing is performed by specialized hardware or software, known as application delivery controllers (ADCs), reverse proxies, or load balancers. These devices analyze incoming requests and distribute them to backend servers based on various factors, such as server health, current load, or user-specific parameters.
5. **Session-based Load Balancing**: In situations where maintaining a user session is important, load balancers can use session-based load balancing. This technique ensures that requests from the same user are consistently directed to the same server throughout their session, to maintain session state and avoid data inconsistencies.
6. **Content-aware Load Balancing**: Load balancers can take into account the content or type of the incoming requests and route them to servers best suited to handle that particular content or request type. For example, requests for images or videos could be directed to servers optimized for media delivery.

Load Balancing



Health Checks

Health checks are a critical aspect of modern distributed systems and microservice architectures. A health check is a monitoring mechanism that allows you to determine the status and availability of individual microservices. It ensures that each service is functioning correctly and can handle incoming requests effectively.

The purpose of health checks is to proactively identify any issues with the services before they impact the overall system's performance and user experience. By periodically checking the health of each microservice, you can prevent cascading failures and improve the resilience of your application.

By implementing robust health checks, you can ensure the stability and reliability of your microservices-based applications, leading to better overall performance and user experience.

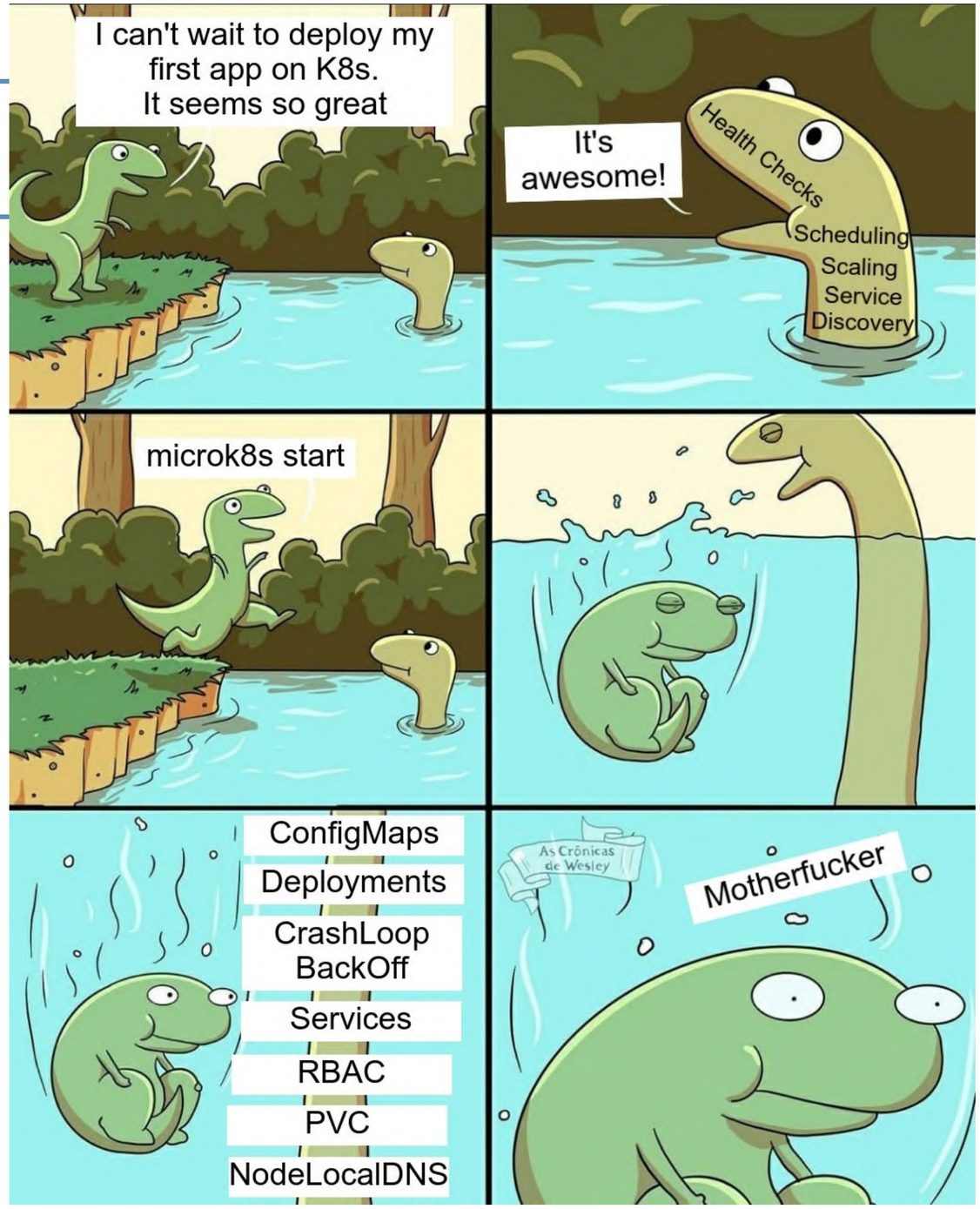
Health Checks

Here are some key points to consider when implementing health checks:

1. **Endpoint for Health Checks**: Each microservice should expose an endpoint specifically dedicated to health checks. This endpoint should return a simple response indicating the service's health status. The response is usually in the form of a JSON object or a plain text message.
2. **Health Status Indicators**: The health check endpoint should provide relevant indicators about the microservice's state. Common indicators include "UP" (healthy) and "DOWN" (unhealthy). Optionally, additional details about the service's state or any potential issues can be included in the response.
3. **Scheduled Health Checks**: The health checks should be performed at regular intervals by a monitoring system or orchestrator. The frequency of health checks can vary based on the requirements of the application and the service's characteristics.
4. **Dependency Health**: A microservice may depend on other services, databases, or external APIs. When performing health checks, it's essential to include checks for these dependencies as well. If a dependency is down or not functioning correctly, it can affect the overall health of the microservice.
5. **Timeouts and Retries**: When performing health checks, set appropriate timeouts and retries to handle potential transient failures. This ensures that false positives are minimized, and the monitoring system can accurately determine the service's health.
6. **Monitoring and Alerting**: Configure a monitoring system to collect health check data and trigger alerts when services are unhealthy. Monitoring tools like Prometheus, Grafana, or DataDog can help track the health status and performance of microservices.
7. **Graceful Degradation**: In cases where a service is temporarily unhealthy but can still function partially, consider implementing a graceful degradation mechanism. This allows the system to continue providing limited functionality during an outage, avoiding complete service failures.
8. **Load Balancer Integration**: If you are using a load balancer to distribute requests among instances of a microservice, integrate the health check results into the load balancing decision-making process. Unhealthy instances should be excluded from receiving new requests until they become healthy again.

DRIM

Nomad vs. Kubernetes



DRIM

Thank you! Questions?

